Rajkiya Engineering College Sonbhadra

Name :
Roll No : 23084101000
Sem : 5th

Sub: DBMS LAB
Branch : CSE, **3rd**year

INDEX

S.N0	Program Tittle	Date	Sign.	Remark
1.	Installing Oracle			
2.	Creating Entity-Relationship Diagram using case tools.			
3.	Writing SQL statements Using ORACLE /MYSQL. A. Writing basic SQL SELECT statements. B. Restricting and sorting data. C. Displaying data from multiple tables. D. Aggregating data using group function. E. Manipulating data. F. Creating and managing tables.			
4.	Creating procedure and functions			
5.	Design and implementation of Student Information System.			
6.	Write a CURSOR to display list of clients in the client Master Table.			
7.	Execute the queries related to Group By and having Clause on tables SALES_ORDER.			
8.	Execute the following queries: A. The NOT NULL B. The UNIQUE Constraint C. The PRIMARY KEY Constraint D. The CHECK Constraint E. Define Integrity Constraints in ALTER table Command			
9.	Execute Nested Queries on tables CLIENT_MASTER, PRODUCT_MASTER, SALESMAN_MASTER, SALES_ORDER, SALES_ORDER_DETAILS.			
10.	Execute queries related to Exists, Not Exists, Union, Intersection, Difference, Join on tables CLIENT_MASTER, PRODUCT_MASTER, SALESMAN_MASTER, SALES_ORDER, SALES_ORDER_DETAILS.			
11.				

Submission Date : _____ **Signature :** _____

01: Installing Oracle

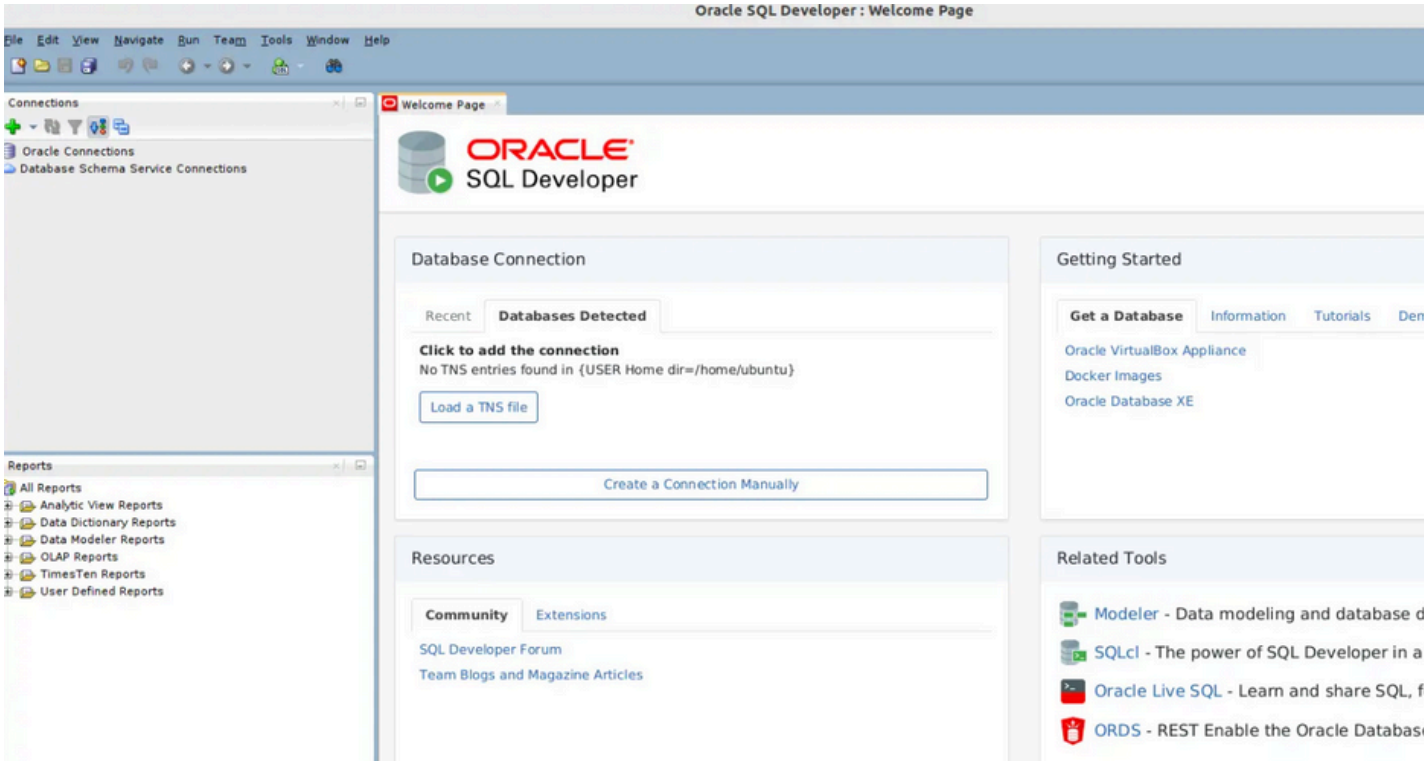
Aim: To install Oracle Database and SQL

Software Requirements:

- Oracle Database (10g / 11g / XE)
- Oracle SQL Developer
- Windows OS

Procedure Requirements:

1. Download Oracle Database setup.
2. Run setup.exe.
3. Choose Create and Configure Database.
4. Select Desktop Class.
5. Set username and password.
6. Complete installation.
7. Open SQL Developer and connect to the database.



02: To create an ER diagram using CASE tools.

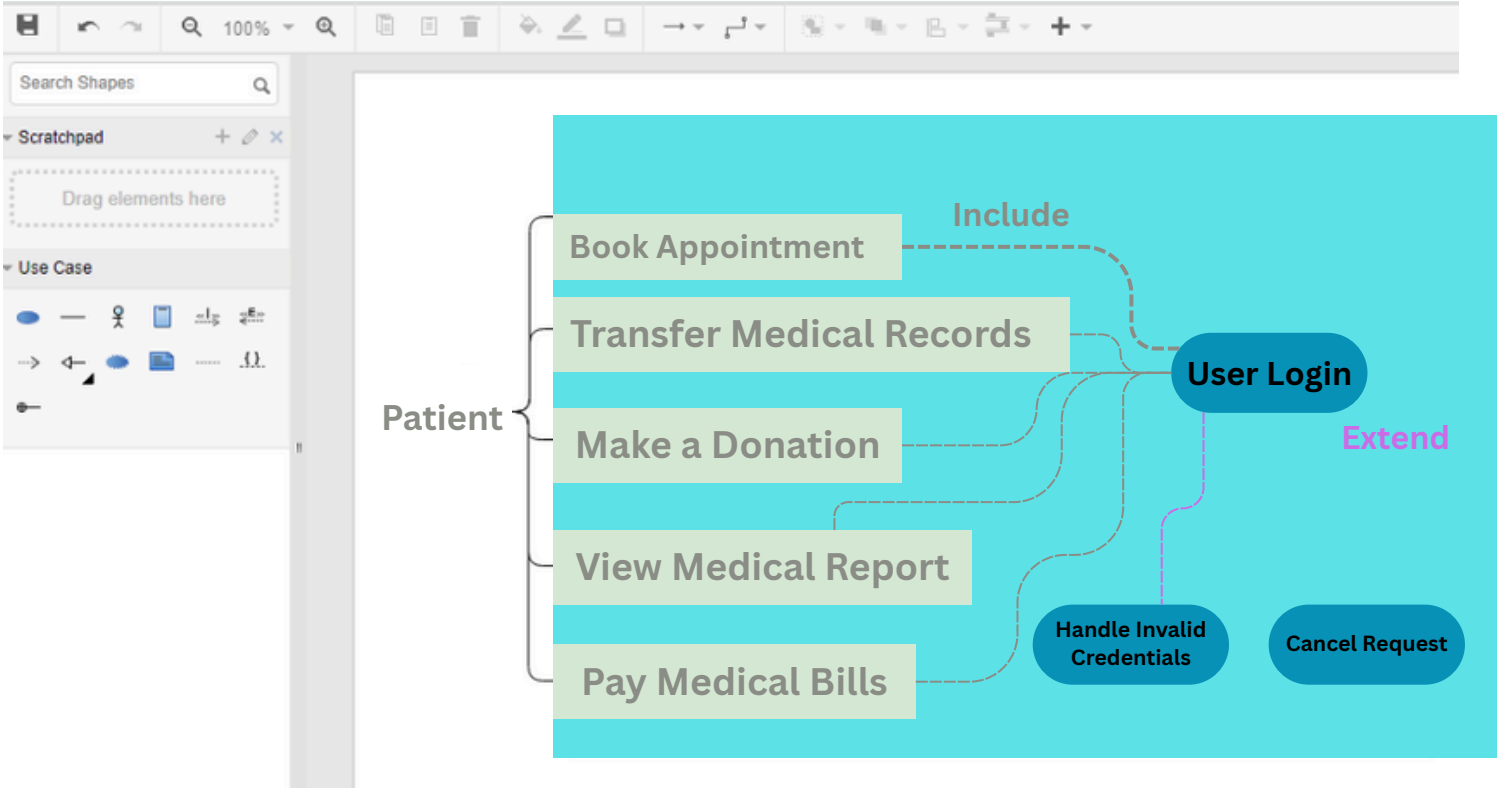
Aim: To create an ER diagram using CASE tools.

Theory:

An ER diagram represents entities, attributes, and relationships between entities.

Procedure Requirements:

1. Open CASE tool (StarUML / ERDPlus).
2. Create entities.
3. Assign attributes and primary keys.
4. Define relationships.
5. Save the diagram.



03: Writing SQL statements Using ORACLE /MYSQL

Aim: To perform basic SQL operations.

A. Writing basic SQL SELECT statements.

```sql
CREATE TABLE CLIENT_MASTER (
    CLIENT_ID INT PRIMARY KEY,
    CLIENT_NAME VARCHAR(100) NOT NULL,
    CITY VARCHAR(100)
);

SELECT * FROM CLIENT_MASTER;
SELECT CLIENT_NAME FROM CLIENT_MASTER;
```

ult Grid | Filter Rows:

CLIENT_ID	CLIENT_NAME	CITY
NULL	NULL	NULL

B. Restricting and sorting data.

```sql
SELECT CLIENT_NAME, CITY
FROM CLIENT_MASTER
WHERE CITY <> 'Mumbai'
ORDER BY CITY DESC;
```

Result Grid | Filter Rows:

CLIENT_NAME	CITY

C. Displaying data from multiple tables..

```sql
CREATE TABLE ORDER_MASTER (
    ORDER_ID INT PRIMARY KEY,
    CLIENT_ID INT,
    ORDER_DATE DATE,
    AMOUNT DECIMAL(10,2),
    FOREIGN KEY (CLIENT_ID) REFERENCES CLIENT_MASTER(CLIENT_ID)
);

SELECT C.CLIENT_NAME, C.CITY, O.ORDER_ID, O.ORDER_DATE, O.AMOUNT
FROM CLIENT_MASTER C
JOIN ORDER_MASTER O ON C.CLIENT_ID = O.CLIENT_ID;
```

Filter Rows: | Export:

_NAME	CITY	ORDER_ID	ORDER_DATE	AMOUNT

D. Aggregating data using group function..

```
SELECT CLIENT_ID, SUM(AMOUNT) AS TOTAL_AMOUNT
FROM ORDER_MASTER
GROUP BY CLIENT ID;
```

CLIENT_ID	TOTAL_AMOUNT
1	7000.00
2	3000.00
3	7000.00
5	4500.00

E. Manipulating data.

```
UPDATE CLIENT_MASTER
SET CITY = 'Mumbai'
WHERE CLIENT_ID = 101;
```

17 19:33:28 UPDATE

F. Creating and managing tables..

```
CREATE TABLE PRODUCT_MASTER (
    PRODUCT_ID INT PRIMARY KEY,
    PRODUCT_NAME VARCHAR(100) NOT NULL,
    PRICE DECIMAL(10,2),
    STOCK INT
);

ALTER TABLE PRODUCT_MASTER
ADD STOCK1 INT;

DROP TABLE PRODUCT_MASTER;
```

PRODUCT_ID	PRODUCT_NAME	PRICE	STOCK	STOCK1
201	Laptop	55000.00	10	NULL
202	Smartphone	25000.00	25	NULL
203	Headphones	2000.00	50	NULL
204	Tablet	30000.00	15	NULL
205	Smartwatch	12000.00	30	NULL
NULL	NULL	NULL	NULL	NULL

04: Creating procedure and functions

Aim: To create stored procedures and functions.

Procedure:

```
DELIMITER $$
CREATE PROCEDURE GetCheapProducts(IN max_price DECIMAL(10,2))
BEGIN
    SELECT PRODUCT_ID, PRODUCT_NAME, PRICE, STOCK
    FROM PRODUCT_MASTER
    WHERE PRICE < max_price
    ORDER BY PRICE ASC;
END $$
DELIMITER ;
CALL GetCheapProducts(20000);
```

Result Grid | Filter Rows: | Export:

PRODUCT_ID	PRODUCT_NAME	PRICE	STOCK
203	Headphones	2000.00	50
205	Smartwatch	12000.00	30

Function :

```
DELIMITER $$
CREATE FUNCTION ProductStockValue(p_id INT)
RETURNS DECIMAL(12,2)
DETERMINISTIC
BEGIN
    DECLARE total_value DECIMAL(12,2);

    SELECT PRICE * STOCK INTO total_value
    FROM PRODUCT_MASTER
    WHERE PRODUCT_ID = p_id;

    RETURN total_value;
END $$
DELIMITER ;

SELECT PRODUCT_NAME, ProductStockValue(PRODUCT_ID) AS StockValue
FROM PRODUCT_MASTER;
```

Result Grid | Filter Rows:

PRODUCT_NAME	StockValue
Laptop	550000.00
Smartphone	625000.00
Headphones	100000.00
Tablet	450000.00
Smartwatch	360000.00

05: Design and implementation of Student Information System.

Aim: To design a Student Information System.

```sql
CREATE TABLE STUDENT_MASTER (
    STUDENT_ID INT PRIMARY KEY,STUDENT_NAME VARCHAR(100) NOT NULL,
    DOB DATE,
    CITY VARCHAR(100)
);

CREATE TABLE COURSE_MASTER (
    COURSE_ID INT PRIMARY KEY,
    COURSE_NAME VARCHAR(100) NOT NULL,
    CREDITS INT
);

CREATE TABLE ENROLLMENT (
    ENROLL_ID INT PRIMARY KEY,STUDENT_ID INT,
    COURSE_ID INT,
    ENROLL_DATE DATE,
    FOREIGN KEY (STUDENT_ID) REFERENCES STUDENT_MASTER(STUDENT_ID),
    FOREIGN KEY (COURSE_ID) REFERENCES COURSE_MASTER(COURSE_ID)
);

CREATE TABLE RESULTS (
    RESULT_ID INT PRIMARY KEY,STUDENT_ID INT,
    COURSE_ID INT,MARKS INT,
    GRADE VARCHAR(2),
    FOREIGN KEY (STUDENT_ID) REFERENCES STUDENT_MASTER(STUDENT_ID),
    FOREIGN KEY (COURSE_ID) REFERENCES COURSE_MASTER(COURSE_ID)
);
```

```sql
INSERT INTO STUDENT_MASTER VALUES(1, 'Amit Sharma', '2003-05-12', 'Delhi'),
(2, 'Neha Verma', '2004-07-20', 'Mumbai'),(3, 'Rajesh Kumar', '2002-11-02', 'Bangalore');

INSERT INTO COURSE_MASTER VALUES(101, 'Database Systems', 4),
(102, 'Operating Systems', 3),(103, 'Computer Networks', 3);

INSERT INTO ENROLLMENT VALUES(1001, 1, 101, '2025-01-10'),
(1002, 1, 102, '2025-01-12'),(1003, 2, 103, '2025-01-15'),
(1004, 3, 101, '2025-01-18');

INSERT INTO RESULTS VALUES(2001, 1, 101, 85, 'A'),
(2002, 1, 102, 78, 'B'),(2003, 2, 103, 92, 'A'),
(2004, 3, 101, 67, 'C');
```

```sql
SELECT S.STUDENT_NAME, C.COURSE_NAME
FROM STUDENT_MASTER S
JOIN ENROLLMENT E ON S.STUDENT_ID = E.STUDENT_ID
JOIN COURSE_MASTER C ON E.COURSE_ID = C.COURSE_ID;
```

esult Grid Filter Rows:

STUDENT_NAME	COURSE_NAME
Amit Sharma	Database Systems
Amit Sharma	Operating Systems
Neha Verma	Computer Network
Rajesh Kumar	Database Systems

06: Write a CURSOR to display list of clients in the client Master Table..

Aim: To use cursor to display client list.

```
DELIMITER $$
CREATE PROCEDURE ShowClients()
BEGIN
    DECLARE done INT DEFAULT 0;
    DECLARE v_client_name VARCHAR(100);
    DECLARE client_cursor CURSOR FOR
        SELECT CLIENT_NAME FROM CLIENT_MASTER;

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
    OPEN client_cursor;

    read_loop: LOOP
        FETCH client_cursor INTO v_client_name;
        IF done = 1 THEN
            LEAVE read_loop;
        END IF;

        SELECT v_client_name AS Client;
    END LOOP;
    CLOSE client_cursor;
END $$
DELIMITER ;
CALL ShowClients();
```

Result Grid

	Client
▶	Vikash Patel

07: Execute the queries related to Group By and having Clause on tables SALES_ORDER..

Aim: To execute GROUP BY and HAVING queries.

```sql
CREATE TABLE SALES_ORDER (
    ORDER_ID INT PRIMARY KEY,
    CLIENT_ID INT,
    ORDER_DATE DATE,
    AMOUNT DECIMAL(10,2)
);

INSERT INTO SALES_ORDER VALUES
(1, 101, '2025-12-01', 5000.00),
(2, 102, '2025-12-02', 3000.00),
(3, 101, '2025-12-03', 7000.00),
(4, 103, '2025-12-04', 2000.00),
(5, 104, '2025-12-05', 9000.00),
(6, 102, '2025-12-06', 4000.00);
```

Using GROUP BY:

```sql
SELECT CLIENT_ID,
SUM(AMOUNT) AS
TOTAL_SALES
FROM SALES_ORDER
GROUP BY CLIENT_ID;
```

esult Grid | Filter Rov

CLIENT_ID	TOTAL_SALES
101	12000.00
102	7000.00
103	2000.00
104	9000.00

Using HAVING:

```sql
SELECT CLIENT_ID, SUM(AMOUNT)
AS TOTAL_SALES
FROM SALES_ORDER
GROUP BY CLIENT_ID
HAVING SUM(AMOUNT) > 6000;
```

esult Grid | Filter Rows:

CLIENT_ID	TOTAL_SALES
101	12000.00
102	7000.00
104	9000.00

08: Execute the following queries:

A. The NOT NULL
B. The UNIQUE Constraint
C. The PRIMARY KEY Constraint
D. The CHECK Constraint
E. Define Integrity Constraints in ALTER table Command

```sql
CREATE TABLE CLIENT_MASTER (
    CLIENT_ID INT PRIMARY KEY,          -- A. PRIMARY KEY (unique + not null)
    CLIENT_NAME VARCHAR(100) NOT NULL,  -- B. NOT NULL (must always have a value)
    EMAIL VARCHAR(100) UNIQUE,          -- C. UNIQUE (no duplicate emails allowed)
    CITY VARCHAR(100),
    CREDIT_LIMIT DECIMAL(10,2)
    CHECK (CREDIT_LIMIT >= 0)           -- D. CHECK (must be non-negative)
);

INSERT INTO CLIENT_MASTER VALUES
(1, 'Amit Sharma', 'amit@example.com', 'Delhi', 50000.00),
(2, 'Neha Verma', 'neha@example.com', 'Mumbai', 30000.00),
(3, 'Rajesh Kumar', 'rajesh@example.com', 'Bangalore', 45000.00);
```

Result Grid Filter Rows:

CLIENT_ID	CLIENT_NAME	CITY
1	Amit Sharma	Delhi
2	Neha Verma	Mumbai
3	Rajesh Kumar	Bangalore
4	Priya Singh	Chennai
5	Vikash Patel	Delhi
NULL	NULL	NULL

09: Execute Nested Queries on tables CLIENT_MASTER, PRODUCT_MASTER, SALESMAN_MASTER, SALES_ORDER, SALES_ORDER_DETAILS.

A. Nested Query with CLIENT_MASTER:

```
SELECT CLIENT_NAME
FROM CLIENT_MASTER
WHERE CLIENT_ID IN (
    SELECT CLIENT_ID
    FROM SALES_ORDER
);
```

Result Grid | Filter

CLIENT_NAME

B. Nested Query with PRODUCT_MASTER:

```
SELECT PRODUCT_NAME, PRICE
FROM PRODUCT_MASTER
WHERE PRICE > (
    SELECT AVG(PRICE)
    FROM PRODUCT_MASTER
);
```

Result Grid | Filter Rows:

PRODUCT_NAME	PRICE
Laptop	55000.00
Smartphone	25000.00
Tablet	30000.00

C. Nested Query with PRODUCT_MASTER:

```
SELECT SALESMAN_NAME
FROM SALESMAN_MASTER
WHERE SALESMAN_ID IN (
    SELECT SALESMAN_ID
    FROM SALES_ORDER
    WHERE AMOUNT > 5000
);
```

Result Grid | F

SALESMAN_NAME
Ravi Kumar
Anita Sharma
Suresh Gupta
Priya Mehta

E. Nested Query with SALES_ORDER

```sql
SELECT ORDER_ID, AMOUNT
FROM SALES_ORDER
WHERE AMOUNT > (
    SELECT MIN(AMOUNT)
    FROM SALES_ORDER
);
```

Result Grid | Filter Rows:

ORDER_ID	AMOUNT
1	5000.00
2	3000.00
3	7000.00
5	9000.00
6	4000.00
NULL	NULL

E. Nested Query with SALES_ORDER_DETAILS

```sql
SELECT PRODUCT_NAME
FROM PRODUCT_MASTER
WHERE PRODUCT_ID IN (
    SELECT PRODUCT_ID
    FROM SALES_ORDER_DETAILS
    WHERE ORDER_ID IN (
        SELECT ORDER_ID
        FROM SALES_ORDER
        WHERE CLIENT_ID = (
            SELECT CLIENT_ID
            FROM CLIENT_MASTER
            WHERE CLIENT_NAME = 'ABC TRN'
        )
    )
);
```

Result Grid | Filter Rows:

PRODUCT_NAME

10: Execute queries related to Exists, Not Exists, Union, Intersection, Difference, Join on tables CLIENT_MASTER, PRODUCT_MASTER, SALESMAN_MASTER, SALES_ORDER, SALES_ORDER_DETAILS.

1. EXISTS

```
SELECT CLIENT_NAME
FROM CLIENT_MASTER C
WHERE EXISTS (
    SELECT 1
    FROM SALES_ORDER S
    WHERE S.CLIENT_ID = C.CLIENT_ID
);
```

Result Grid | Filter

CLIENT_NAME

2. NOT EXISTS

```
SELECT CLIENT_NAME
FROM CLIENT_MASTER C
WHERE NOT EXISTS (
    SELECT 1
    FROM SALES_ORDER S
    WHERE S.CLIENT_ID = C.CLIENT_ID
);
```

Result Grid

CLIENT_NAME
Amit Sharma
Neha Verma
Rajesh Kumar
Priya Singh

3. UNION

```
SELECT CITY FROM
CLIENT_MASTER
UNION
SELECT CITY FROM
SALESMAN_MASTER;
```

Result Grid

CITY
Delhi
Mumbai
Bangalore
Chennai

4. INTERSECTION (using IN)

```
SELECT CITY
FROM CLIENT_MASTER
WHERE CITY IN (
    SELECT CITY
    FROM SALESMAN_MASTER
);
```

Result Grid

CITY
Delhi
Mumbai
Bangalore
Chennai
Delhi

5. DIFFERENCE (using NOT IN)

```
SELECT CITY
FROM CLIENT_MASTER
WHERE CITY NOT IN (
    SELECT CITY
    FROM SALESMAN_MASTER
);
```

Result Grid Filter

CITY

6. JOIN

```
SELECT C.CLIENT_NAME,
S.ORDER_ID, P.PRODUCT_NAME,
D.QUANTITY
FROM CLIENT_MASTER C
JOIN SALES_ORDER S ON
C.CLIENT_ID = S.CLIENT_ID
JOIN SALES_ORDER_DETAILS D ON
S.ORDER_ID = D.ORDER_ID
JOIN PRODUCT_MASTER P ON
D.PRODUCT_ID = P.PRODUCT_ID;
```

Result Grid Filter Rows: Export:

CLIENT_NAME	ORDER_ID	PRODUCT_NAME	QUANTITY